

Exact Circular Pattern Matchings Using Bit-Parallelism and q -Gram Technique*

Kuei-Hao Chen, Guan-Shieng Huang[†] and Richard Chia-Tung Lee

Department of Computer Science and Information Engineering,
National Chi Nan University, Puli, Nantou, Taiwan.

{s95321902, shieng, rctlee}@ncnu.edu.tw

ABSTRACT

We present three efficient algorithms for exact circular string matching. One of the algorithms is for single circular pattern and the others are for multiple circular patterns. Our algorithms apply q -grams and bit parallelism. The algorithms are given names $CBNDM_q$, $CMultiBNDM$ and $CMultiBNDM_q$, respectively. These two problems can also be solved by some proposed multiple patterns matching algorithms. We use q -grams in a different way in order to improve filtration efficiency. We ran some experiments to verify that the performances of these algorithms are better than other bit-parallelism algorithms without q -grams.

1 Introduction

The classical pattern matching problem is to find all occurrences of a pattern P of length m in a text T of length n . In this paper, we consider exact circular string matching (CPM) problem [1]. Given a pattern $P = p_1p_2 \cdots p_m$, a string $P^{(i)} = p_ip_{i+1} \cdots p_m p_1 \cdots p_{i-1}$, for $1 \leq i \leq m$, is a circular string of P . In CPM, given a text T and a pattern P , we want to find all occurrences of the circular patterns $P^{(i)}$ in the text T , for $1 \leq i \leq m$. Some special case applications of this problem, for example, in image processing analysis, or in Bacterial and mitochondrial DNA analysis can be found in [2]. In some studies, the circular patterns $P^{(i)}$ of P are called *conjugates* of P [3–6].

A number of solutions to solve the CPM problem have been presented in [4–7]. There exists a linear time algorithm to solve the CPM problem based on the suffix tree [5]. The $CBNDM$ (Circular Backward Nondeterministic Dawg Matching) algorithm is a bit-parallel algorithm and a variation of the $BNDM$ algorithm [7, 8]. In addition, some algorithms address the indexing version of the problem [4, 6].

The CPM problem can be transformed to the multi-pattern matching problem. For a circular pattern P of length m , we produce m patterns $P^{(1)}, P^{(2)}, \dots, P^{(m)}$ from P . Then, it can be solved by some proposed multi-pattern matching algorithms such as Aho-Corasick [10], Commentz-Walter [11], Multiple Shift-And [12], Wu and Manber [13], MultiBNDM [8, 9], MultiBDM [14, 15], TARA [16], Dawg-Match [17], BG [18], AOAC [19] algorithms.

Multiple (exactly) circular pattern matching (MCPM for short) problem is an extension of the CPM problem where the input is a set of r equal-length patterns $P_1, P_2, \dots, P_r$ whose lengths are all m and a text T of length n . The problem is finding all occurrences of $P_j^{(i)}$, for $1 \leq i \leq m, 1 \leq j \leq r$, in T . This problem arises in many applications.

Our main contribution is as follows. We will present three algorithms: $CBNDM_q$ for single circular pattern matching; $CMultiBNDM$ and $CMultiBNDM_q$ for multiple circular patterns matching. The $CBNDM_q$ and $CMultiBNDM_q$ algorithms are filtering-based algorithms and operate in three phases. The first phase preprocesses the pattern(s). The second phase quickly filters out impossible candidates and leaves out potential candidates for the third phase that does a final checking. All algorithms apply the bit-parallelism.

The organization of this paper is as follows.

*This work was supported in part by the National Science Council of the Republic of China, Taiwan (grant No. NSC 100-2221-E-260-026).

[†]To whom correspondence should be addressed.
Email:shieng@ncnu.edu.tw

The basic idea of the reverse-factor algorithm and the BNBM algorithm are introduced in Sect. 2 and Sect. 3, respectively. They are the basics of the proposed algorithms in this paper. Section 4 introduces the CBNBM algorithm. Section 5 describes our main algorithm, named CBNBM_q, for single circular pattern matching. Sections 6 and 7 introduce our next two algorithms for multiple circular patterns matching. The time complexity of all algorithms proposed in this paper are analyzed in Section 8. Several useful experimental results are shown in Section 9. Finally, the conclusion ends in Section 10.

2 The Basic Idea of the Reverse-Factor Algorithm

The reverse-factor algorithm was proposed in [20]. Before introducing the algorithm, we need to define a term, called *proper suffix*. A proper suffix of a string P is a suffix of P which is not exactly P . The basic principles of this algorithm can be illustrated as follows:

1. Given a text string T of length n and a pattern string P of length m , we open a window W of size m in T . If W and P are equal, we report that an exact match is found here.
2. Let $LPSP(W, P)$ denote the longest proper suffix of W which is equal to a prefix of P . We shift right the window W for $m - |LPSP(W, P)|$ places.

To find $LPSP(W, P)$, we may build a suffix tree of the reverse of P in the preprocessing phase and scan the window W from right to left in the searching phase. However, the suffix tree approach to solve the reverse-factor algorithm is not easy to implement. Furthermore, in special applications, several simple alternatives usually have better performance than the suffix-tree approach in practice. Therefore, Navarro and Raffinot proposed a bit-parallelism approach named BNBM [8] to simplify the reverse-factor algorithm.

3 Single Pattern Matching Solved by the BNBM Algorithm

To conveniently discuss the bit-parallelism technique, we use the following notations. Exponentiation stands for bit repetition (e.g. $0^31 = 0001$). For a string P , we use the notation $P =$

$P[1, m]$ to mean that P is a string of length m , starting from position 1. We also use $P[i, j]$ to denote the substring of P from position i to position j for $1 \leq i \leq j \leq m$. In particular, $P[1, j]$ is called a *prefix* of P and $P[i, m]$ is a *suffix* of P . We use $P[i]$ to denote the i th character of P . In the bit operators, “&” represents bitwise-AND operation and “|” represents bitwise-OR operation. The shift left operation “ $\ll$ ” removes the highest bit, moves the other bits left for one position and enters a zero from the right. The circular shift left operation “ $\lll$ ” rotates a vector one bit left and fills its rightmost bit to be first bit.

To use the bit-parallelism approach, we first define an *incidence matrix* B of σ by m where σ is the alphabet size. Let $B[\alpha]$ be a row of B that corresponds to character α , which is set to be 1 at the i -th position if and only if $P[i] = \alpha$. For example, assume that $P = \text{tgacc}$. We then have $B[a] = (00100)$, $B[c] = (00011)$, $B[g] = (01000)$ and $B[t] = (10000)$.

For an attempt of the scan, we open a window W of size m which corresponds to $T[i, i + m - 1]$ for $1 \leq i \leq n - m + 1$. For the j -th iteration in this attempt, let $D_j = D_j[1, m]$ be a bit vector. The bit $D_j[k]$ is set to be 1 if and only if there exists a substring $P[k, k + j - 1]$ which is equal to suffix $W[m - j + 1, m]$. In particular, if $D_j[1] = 1$, it indicates that we have found a prefix $P[1, j]$ of P which is equal to a suffix $W[m - j + 1, m]$ of W . This particular information is very important. We shall use a parameter *last* to record this information. That is, if $D_j[1] = 1$, we set $last = j$.

Initially, we set $D_0 = 1^m$ and $last = 0$. We scan the window W from right to left. When $W[m - j + 1]$ was scanned at the j -th iteration, we update D_j based on the following rule:

$$D'_j = (D_{j-1} \& B[W[m - j + 1]]) \quad (1)$$

- If $D'_j = 0^m$, we stop the scanning. Then we know that $|LPSP(W, P)| = last$ and thus we shift W right for $m - last$ places.
- If $D'_j \neq 0^m$ and $j < m$, we check whether $D'_j[1] = 1$. We set $last$ to be j if $D'_j[1] = 1$, and take no action if otherwise. After checking whether $D'_j[1] = 1$, we shift vector D'_j one bit left, fill the rightmost bit to be 0 and store this value to D_j as

$$D_j = D'_j \lll 1 \quad (2)$$

and we continue the next iteration.

- If $D'_j \neq 0^m$ and $j = m$, it means that we have found an exact match of P in T . We then

stop this attempt of scan, report a match and shift W right for $S = m - \text{last}$ places.

Algorithm 1 shows a complete description of the BNBM algorithm for single pattern matching. Notice that in Algorithm 1, we drop the subscript j in variables D_j for the purpose to reuse memory.

Algorithm 1: The BNBM algorithm for single pattern matching

Input: Two strings $T = T[1, n]$ and $P = P[1, m]$
Output: The locations of all occurrences of P in T

```

1 Begin
2   Set  $i = 1$ .
3   Compute the incidence matrix  $B$  of  $P$ .
4   While  $i \leq (n - m + 1)$  do
5      $j = 0$ .
6      $\text{last} = 0$ .
7      $D = 1^m$ .
8     While  $D \neq 0^m$  do
9        $j = j + 1$ .
10       $D = (D \ll 1) \& B[T[i + m - j]]$ .
11      If  $j = m$  and  $D[1] = 1$  then report an
        occurrence of  $P$  at  $T[i]$  and exit the
        inner loop.
12      If  $D[1] = 1$  then  $\text{last} = j$ .
13    End of while
14     $i = i + (m - \text{last})$ .
15  End of while
16 End
```

4 Circular Pattern Matching Solved by the CBNDM Algorithm

The CBNDM algorithm is based upon the BNBM algorithm (see Algorithm 1). For an attempt of scan, we open a window $W[1, m] = T[i, i + m - 1]$ for $1 \leq i \leq n - m + 1$. The incidence matrix B is still used. For the j -th iteration in the current attempt, let $D_j = D_j[1, m]$ be a bit vector. The bit $D_j[k]$ is set to be 1 if and only if $P^{(k)}[1, j]$ is equal to $W[m - j + 1, m]$ for $1 \leq k, j \leq m$. For example, suppose that $W = \text{agacttc}$ and $P = \text{cagactt}$. We have $P^{(5)} = P[5, 7]P[1, 4] = \text{cttcaga}$. Then $D_4 = (0000100)$ because $P^{(5)}[1, 4] = W[4, 7]$.

Initially, we set $D_0 = 1^m$. We open a window W of length m which starts at $T[i]$. We scan W from right to left. For the j -th iteration, we read $T[i + m - j]$ and perform an AND operation on B and D_{j-1} based on Equation (3):

$$D_j = (D_{j-1} \xrightarrow{\ll} 1) \& B[T[i + m - j]] \quad (3)$$

- If $D_j = 0^m$ for some j , we stop the scanning. Then we shift W right for $S = m - (j - 1)$ places and continue the next attempt.
- If $D_j \neq 0^m$, $j < m$ and $D_j[k] = 1$ for some k , it indicates that $W[m - j + 1, m]$ is equal to $P^{(k)}[1, j]$. We simply continue the next iteration.
- If $D_j \neq 0^m$, $j = m$ and $D_j[k] = 1$ for some k , it means that we have found an exact match of $P^{(k)}$ in T . We then stop the iteration, report a match and shift W right for 1 place.

Algorithm 2 shows a complete description of the CBNDM algorithm.

Algorithm 2: Circular pattern matching using the CBNDM algorithm

Input: Two strings $T[1, n]$ and $P[1, m]$.
Output: The locations of all occurrences of $P^{(i)}$ for $1 \leq i \leq m$ in T

```

1 Begin
2   Compute the incidence matrix  $B$  of  $P$ .
3   Set  $i = 1$ .
4   While  $i \leq (n - m + 1)$  do
5      $j = 0$ .
6      $D = 1^m$ .
7     While  $D \neq 0^m$  do
8        $j = j + 1$ .
9        $D = (D \xrightarrow{\ll} 1) \& B[T[i + m - j]]$ .
10      If  $D \neq 0^m$ ,  $j = m$  and  $D[k] = 1$  then
        report an exact match of  $P^{(k)}$  at  $T[i]$ 
        and exit the inner loop.
11      End of while
12       $i = i + (m - j + 1)$ .
13    End of while
14 End
```

5 Circular Pattern Matching Solved by the CBNDM_q Algorithm

The q -gram technique is useful for speeding up the computation of BNBM algorithm [18], as well as for CBNDM (cf. Algorithm 2). We explain this technique directly on CBNDM. Recall that in each attempt of CBNDM, it scans characters of W one by one from right to left in order to make sure that the current scanned suffix of W is a factor of $P^{(i)}$ for some i . If it does, then the scan continues until a match is found; otherwise this attempt fails and the window W is shifted. The idea of the q -gram technique is as follows. We pre-compute a hash table named Gram_s to store the results of vector D for the first q iterations of every attempt of

the scan. More precisely, all length- q subwords of $P^{(i)}$ for $1 \leq i \leq m$ serve as the keys of Gram_S . In the beginning of scanning W , we read $W[m - q + 1, m]$ at once and check whether it appears in Gram_S . If it does, we retrieve its corresponding vector D from Gram_S and continue the scan of W from position $m - q$ to position 1; otherwise we shift W right for $m - q + 1$ places because $W[m - q + 1, m]$ is not a factor of $P^{(i)}$ for $1 \leq i \leq m$.

For example, suppose that $P = \text{acggtta}$ and $q = 2$. Consider the 2-gram ac . Since $B[\text{a}] = (1000001)$ and $B[\text{c}] = (0100000)$, we have $D = (B[\text{c}] \ll 1) \& (B[\text{a}]) = (1000000)$, which is stored in the hash table Gram_S and corresponds to key ac .

Algorithm 3 shows a complete description of the CBNDM_q algorithm.

Algorithm 3: Circular pattern matching using the CBNDM_q algorithm

Input: Two strings $T[1, n]$ and $P[1, m]$.
Output: The locations of all occurrences of $P^{(i)}$ where $1 \leq i \leq m$ in T

```

1 Begin
2   Compute the incidence matrix  $B$  of  $P$ .
3   Construct the hash table  $\text{Gram}_S$ .
4   Set  $i = 1$ .
5   While  $i \leq (n - m + 1)$  do
6     Set  $j = q$ .
7      $D = \text{Gram}_S[T[i + m - q, i + m - 1]]$ .
8     While  $D \neq 0^m$  do
9        $j = j + 1$ .
10       $D = (D \ll 1) \& B[T[i + m - j]]$ .
11      If  $D \neq 0^m, j = m$  and  $D[k] = 1$  then
        report an exact match of  $P^{(k)}$  at  $T[i]$ 
        and exit the inner loop.
12    End of while
13     $i = i + (m - j + 1)$ .
14  End of while
15 End
    
```

How do we determine the value of q so that the execution time can be minimized? We propose the following solution. Suppose that W and P are uniformly distributed random strings. Notice that the overhead of the q -gram technique is almost propositional to q . Hence, we use $q \times T_h$ to model this overhead. This includes reading $W[m - q + 1, m]$, hashing and retrieving D . Next, we consider the cost of processing one window in the searching phase of CBNDM . Let T_b be the overhead of processing one character of W . Notice that T_b is indeed a function of $\lceil m/w \rceil$ where w is the number of bits in a machine word. Let c_t be the probability that $W[m - t + 1]$ will be read by CBNDM . Then the average time taken by

CBNDM_q can be formulated as

$$f(q) = qT_h + \sum_{t=q+1}^m c_t T_b \text{ for } 1 \leq q \leq m. \quad (4)$$

The first term qT_h counts for Line 7 and the second term $\sum_{t=q+1}^m c_t T_b$ counts for Lines 8 to 12 of Algorithm 3. To find the optimal q that can minimize Equation 4, we first take its backward difference and get

$$\nabla f(q) = f(q) - f(q - 1) = T_h - c_q T_b. \quad (5)$$

Notice that

$$c_1 = 1 \geq c_2 \geq \dots \geq c_m > 0.$$

Furthermore, T_h is always less than T_b ; otherwise the q -gram technique would become useless. Hence, $\nabla f(1) = T_h - T_b < 0$ and $\nabla f(q)$ is increasing as q increasing. Therefore, the largest q that satisfies $\nabla f(q) \leq 0$ is the optimal solution.

We can easily show that the value of q should be no more than $\lceil (2 + \epsilon) \log_\sigma m \rceil$ for $0 \leq \epsilon < 1$. Notice that $c_t \leq (m/\sigma^{t-1})$ for $2 \leq t \leq m$, because $W[m - t + 2, m]$ might occur in at most m places of P . Let $t = \lceil (2 + \epsilon) \log_\sigma m \rceil$. Then $c_{t+1} \leq (m/\sigma^t) \leq (1/m^{1+\epsilon})$. Hence $\nabla f(t + 1) > T_h - (T_b/m^{1+\epsilon}) > 0$ as m becomes sufficiently large. Therefore, the optimal q should be no more than $\lceil (2 + \epsilon) \log_\sigma m \rceil$.

6 Multiple Circular Pattern Matching Solved by the CMultiBNDM Algorithm

In this section, we extend CBNDM to multiple circular pattern matching. Given a set P of r patterns $P_1 = P_1[1, m], P_2 = P_2[1, m], \dots, P_r = P_r[1, m]$ and a text $T = T[1, n]$, let $P_j^{(i)} = P_j[i, m]P_j[1, i - 1]$, for $1 \leq i \leq m, 1 \leq j \leq r$. Our problem is to find all occurrences of $P_j^{(i)}$ in T . Let us consider $T = \text{atacc}$, $P = \{P_1 : \text{actg}, P_2 : \text{ctaa}\}$. We open the window $W[1, 4] = T[1, 4] = \text{atac}$ and scan it from right to left. We first read $W[4] = \text{c}$ and find that it occurs in $P_1[2]$ and $P_2[1]$. When we read the next character $W[3] = \text{a}$, we check whether it is $P_1[1]$ and $P_2[4]$. In our case, $P_1[1]$ does. We then read the next character $W[2] = \text{t}$ and check whether it is identical to $P_1[4]$.

Our new algorithm for MCPM is named CMultiBNDM . We first concatenate all patterns

as $X = P_1 P_2 \cdots P_r = X[1, rm]$. For example, if $P = \{P_1 : \text{gtc}, P_2 : \text{acc}, P_3 : \text{tca}\}$. Then $X = \text{gtcacctca}$.

The preprocessing phase of CMultiBNDM is similar to CBNDM. Incidence vector B with alphabet size σ by rm is used.

For an attempt of the scan, we open a window $W = W[1, m]$ located at $T[i, i + m - 1]$, $1 \leq i \leq n - m + 1$. For the j -th iteration, we read character $T[i + m - j]$. Let $D_j = D_j[1, rm]$ be a bit vector. The bit $D_j[a]$ is in D_j set 1 if and only if $P_t[k, k + j - 1] = T[i + m - j, i + m - 1]$ where $t = \lceil a/m \rceil$ and $k = a - m(t - 1)$. This means that there exists a prefix $P_t^{(k)}[1, j]$ which is equal to the suffix $W[m - j + 1, m]$ located at $T[i + m - j, i + m - 1]$. To maintain D_j , we left-rotate each m -bit field of D_j respectively. For each m -bit field, it shifts one bit to the left, and fill its rightmost bit to be the old first bit. For example, assume that $D_j = (100011100)$ and $m = 3$. The first 3-bit field is (100), after left-rotating, we have (001). After left-rotating all 3-bit fields, we have $D_j = (001110001)$. In the bit-parallelism version, the rotating operations of all m -bit fields can be simultaneously processed. Let $DF = (10^{m-1})^r$ which is bit one in the highest bit of each m -bit field and the bit mask $CL = (1^{m-1}0)^r$ which is to solve the problem that the highest bit corresponding to P_t becomes the lowest bit of P_{t-1} . We perform the following operation:

$$D'_j = ((D_j \& DF) \gg (m-1)) \mid ((D_j \ll 1) \& CL) \quad (6)$$

Initially, we set $D_0 = 1^{rm}$ and perform the left-rotation operation of D_0 based on Equation 6. We scan the window which starts at $T[i]$ from right to left. In the j -th iteration, when a new character $T[i + m - j]$ was scanned, we update D_j based on Equation 7:

$$D_j = D_{j-1} \& B[T[i + m - j]] \quad (7)$$

- If $D_j = 0^{rm}$ for some j , we stop this attempt of scanning. Then we shift W right for $S = m - (j - 1)$ places.
- If $D_j \neq 0^{rm}$, $j < m$ and $D_j[m(t-1) + k] = 1$ for $1 \leq t \leq r$ and $1 \leq k \leq m$, it means that there exists prefix $P_t^{(k)}[1, j]$ which is equal to suffix $W[m - j + 1, m]$. We simply continue the next iteration.
- If $D_j \neq 0^{rm}$, $j = m$ and $D_j[m(t-1) + k] = 1$, we have found an exact match of $P_t^{(k)}$ in $T[i, i + m - 1]$. Then report a match and shift W right for 1 place.

Algorithm 4 shows a complete description of the CMultiBNDM algorithm.

Algorithm 4: Circular pattern matching using the CMultiBNDM algorithm

Input: A strings $T[1, n]$ and a set of patterns $P_t[1, m]$ for $1 \leq t \leq r$.
Output: The locations of all occurrences of $P_t^{(k)}$ in T for $1 \leq k \leq m$ and $1 \leq t \leq r$.

```

1 Begin
2   Set  $X = P_1 P_2 \cdots P_r$ .
3   Compute the incidence matrix  $B$  of  $X$ .
4   Set  $CL = (1^{m-1}0)^r$ .
5   Set  $DF = (10^{m-1})^r$ .
6   Set  $i = 1$ .
7   While  $i \leq (n - m + 1)$  do
8      $D = 1^{rm}$ .
9      $j = 0$ .
10    While  $D \neq 0^{rm}$  do
11       $D = ((D \& DF) \gg (m-1)) \mid ((D \ll 1) \& CL)$ .
12       $j = j + 1$ .
13       $D = D \& B[T[i + m - j]]$ .
14      If  $D \neq 0^{rm}$ ,  $j = m$  and  $D[m(t-1) + k] = 1$  then report an exact match of  $P_t^{(k)}$  at  $T[i]$  and exit the inner loop.
15    End of while
16     $i = i + (m - j + 1)$ .
17  End of while
18 End

```

7 Multiple Circular Pattern Matching Solved by the CMultiBNDM_q Algorithm

CMultiBNDM_q combines CBNDM_q and CMultiBNDM to solve the MCPM problem. In the preprocessing, incidence vector B , bit vector D , bit masks DF and CL are still used. The q -gram hash table, now called **Gram_M** for the MCPM problem, can be built in a similar way as **Gram_S** for the CPM problem.

Algorithm 5 shows a complete description of the CMultiBNDM_q algorithm.

We give a complete example. Suppose that $T = \text{acacttgcttc}$, $P = \{P_1 : \text{gctaca}, P_2 : \text{gtactt}\}$ and $q = 3$. The corresponding incidence matrix B and hashing table **Gram_M** are shown in Figure 1.

Initially, $j = q = 3$. In the first attempt, we scan the characters of window $W = T[1, 6] = \text{acactt}$ from right to left. In the filtering phase, the rightmost $q = 3$ characters of W is $T[4, 6] = \text{ctt}$ and **Gram_M**[**ctt**] = $(0^9 10^2)$, which means **ctt** occurs in some circular pattern. Thus we enter the checking phase to scan the remaining string

Algorithm 5: Circular pattern matching using the CMultiBNDM_q algorithm

Input: A strings $T[1, n]$ and a set of patterns $P_t[1, m]$ for $1 \leq t \leq r$.

Output: The locations of all occurrences of $P_t^{(k)}$ in T for $1 \leq k \leq m$ and $1 \leq t \leq r$.

```

1 Begin
2   Set  $X = P_1 P_2 \cdots P_r$ .
3   Compute the incidence matrix  $B$  of  $X$ .
4   Set  $CL = (1^{m-1}0)^r$ .
5   Set  $DF = (10^{m-1})^r$ .
6   Construct the hash table  $\text{Gram}_M$ .
7   Set  $i = 1$ .
8   While  $i \leq (n - m + 1)$  do
9     Set  $j = q$ .
10     $D = \text{Gram}_M[T[i + m - q, i + m - 1]]$ .
11    While  $D \neq 0^r$  do
12       $D = ((D \& DF) \gg (m - 1)) \mid ((D \ll 1) \& CL)$ .
13       $j = j + 1$ .
14       $D = D \& B[T[i + m - q - j]]$ .
15      If  $D \neq 0^r$ ,  $j = m$  and
         $D[m(t - 1) + k] = 1$  then report an
        exact match of  $P_t^{(k)}$  at  $T[i]$  and exit the
        inner loop.
16    End of while
17     $i = i + (m - j + 1)$ .
18  End of while
19 End
    
```

$T[1, 3] = \text{aca}$.

In the checking phase, initially, $D_3 = (0^9 10^2)$. We start with the character $T[3] = \text{a}$. In the first iteration, $j = 4$, the processing is shown in Figure 2.

In Figure 2, after calculating, we found that $D_4 = (0^8 10^3)$. In D_4 , $D_4[9] = 1$. This means that $T[3, 6]$ is equal to the prefix of $P_2^{(3)} = \text{actt}$ of length $j = 4$. We continue the next iteration to read next character of W .

In the second iteration, $j = 5$, we have $D_4 = (0^8 10^3)$ and $T[2] = \text{c}$. After a circular bit shift operation and an AND operation, we found that $D_5 = (0^{12})$ containing all 0's. It means that the suffix cactt does not occur in any circular pattern. We stop the attempt. The shift is $m - j + 1 = 6 - 5 + 1 = 2$, which is shown in Figure 3.

In the second attempt, we scan the characters of window $W = T[3, 8] = \text{acttgt}$ from right to left. In the filtering phase, the rightmost $q = 3$ characters $T[6, 8] = \text{tgt}$ and $\text{Gram}_M[\text{tgt}] = (0^{11} 1)$. We enter the checking phase to scan string $T[3, 5] = \text{act}$ from right to left.

In the checking phase, we start with the character $T[5] = \text{t}$ and $j = 4$. The processing for the first iteration of the second attempt is shown in Figure 4.

In Figure 4, we found that $D_4 = (0^{10} 10)$. In

```

T = acacttggttc
P = {gctaca, gtactt}

B[a] = (000101001000)  GramM[gct] = (100000000000)
B[c] = (010010000100)  GramM[cta] = (010000000000)
B[g] = (100000100000)  GramM[tac] = (001000010000)
B[t] = (001000010011)  GramM[aca] = (000100000000)
                        GramM[cag] = (000010000000)
                        GramM[agc] = (000001000000)
                        GramM[gta] = (000000100000)
                        GramM[act] = (000000001000)
                        GramM[ctt] = (000000000100)
                        GramM[ttg] = (000000000010)
                        GramM[tgt] = (000000000001)
    
```

Figure 1: Memory dump of CMultiBNDM_q after the preprocessing phase of an example.

The 1st iteration

W	
$T = \text{acactt} \underline{\text{g}} \text{gttc}$	
$D_3 = 000000000100$	$D_3 = 000000000100 \ll 1$
$\& DF = 100000100000$	0000000001000
$000000000000 \gg 5$	$\& CL = 111110111110$
000000000000	0000000001000
0000000001000	
$D'_3 = 0000000001000$	
$\& B[a] = 000101001000$	
$D_4 = 0000000001000$	

Figure 2: The first iteration in the first attempt.

The 2nd iteration

W	
$T = \text{acactt} \underline{\text{g}} \text{gttc}$	
$D_4 = 0000000001000$	$D_4 = 0000000001000 \ll 1$
$\& DF = 100000100000$	0000000001000
$000000000000 \gg 5$	$\& CL = 111110111110$
000000000000	0000000001000
0000000001000	
$D'_4 = 0000000001000$	
$\& B[c] = 010010000100$	
$D_5 = 0000000000000$	

Figure 3: The second iteration in the first attempt.

The 1st iteration

W	
$T = \text{acactt} \underline{\text{g}} \text{gttc}$	
$D_3 = 0000000000001$	$D_3 = 0000000000001 \ll 1$
$\& DF = 100000100000$	0000000000010
$000000000000 \gg 5$	$\& CL = 111110111110$
000000000000	0000000000010
0000000000010	
$D'_3 = 0000000000010$	
$\& B[t] = 001000010011$	
$D_4 = 0000000000010$	

Figure 4: The first iteration in the second attempt.

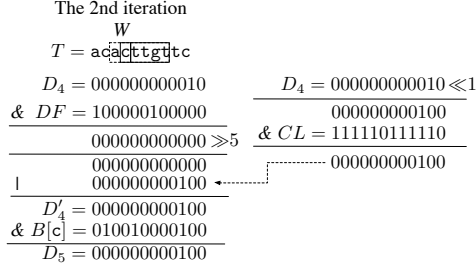


Figure 5: The second iteration in the second attempt.

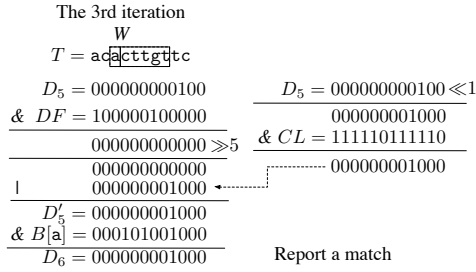


Figure 6: The third iteration in the second attempt.

$D_4, D_4[11] = 1$. This means that the suffix $T[5, 8]$ is equal to the prefix of $P_2^{(5)} = \text{tgtt}$ of length 4. We continue the next iteration.

In the second iteration, $j = 5$, we have $D_4 = (0^{10}10)$ and the character of $T[4]$ is **c**. After computing, we found that the bits in $D_5 = (0^910^2)$ are not all 0's. This means that $T[4, 8]$ is equal to the prefix of $P_2^{(4)} = \text{cttgtt}$ of length 5. We continue the next iteration. The result is shown in Figure 5.

In the third iteration, $j = 6$, we have $D_5 = (0^910^2)$ and the character $T[3] = \text{a}$. After computing, we found that the bits in $D_6 = (0^810^3)$ are not all 0's. We report the existence of an exact match. Then the shift is 1, which is shown in Figure 6.

In the third attempt, we scan the characters of window $W = T[4, 9] = \text{cttgtt}$ from right to left. In the filtering phase, the rightmost q characters $T[7, 9] = \text{gtt}$ and $\text{Gram}_M[\text{gtt}] = (0^{12})$ contains all 0's. It means that the suffix **gtt** does not occur in any circular pattern. Thus, the shift is $m - j + 1 = 4$. After shifting, the boundary of the window is larger than n . We stop.

8 The Time-Complexity Analysis

For the time-complexity analysis, we first consider CBNDM_q . Assume that the alphabet size σ is a constant. The algorithm has three phases: pre-processing, filtering and checking. When $m \leq w$ where w is the number of bits in a computer word, in the preprocessing phase, constructing the incidence matrix B and the hash table Gram_S needs $O(m \log_\sigma m)$ time.

In the searching phase, the size of a window is m , which is equal to the length of the pattern. In the best case, no match candidate in the Gram_S is found. It means that the checking phase is not needed and then the shift is $m - \lceil 2 \log_\sigma m \rceil + 1$ places in the each attempt because $q = \lceil 2 \log_\sigma m \rceil$. In this case, the number of comparisons is $O(n \log_\sigma m/m)$. In the worst case, there are $O(m)$ candidates which are matched and then need to check remaining $m - \lceil 2 \log_\sigma m \rceil$ characters in the checking phase. The number of comparisons becomes $O(mn)$.

Consider another case $m > w$. In the preprocessing phase, the CBNDM_q algorithm needs $O(m \log_\sigma m \times \lceil m/w \rceil)$ comparisons since it needs to work on $\lceil m/w \rceil$ computer words, and then the checking time is $O(nm^2/w)$. Therefore, the total time of the CBNDM_q algorithm is $O(nm^2/w)$.

Let us consider the time complexities of the CMultiBNDM and CMultiBNDM_q algorithms. Let us assume that $rm > w$ holds. The worst case complexity of the basic MultiBNDM algorithm is $O(nm \lceil rm/w \rceil)$. We directly apply the main idea of the MultiBNDM algorithm into these two algorithms. In the CMultiBNDM algorithm, the time complexity of the preprocessing phase is $O(rm \lceil rm/w \rceil)$. In the CMultiBNDM_q algorithm, the time complexity of the preprocessing phase is $O(\min\{\lceil \log_\sigma rm \rceil, m\} \times rm \times \lceil rm/w \rceil)$. The checking phase of the both algorithms is similar and they work in $O(nm \lceil rm/w \rceil)$.

9 Experimental Results

In order to verify the good practical behavior of the proposed algorithms, we ran the experiments on several algorithms. The experiments were made on a computer with an Intel Pentium 4 processor of 3GHz, with 1Gb of RAM and a computer word is 32 bits, under Fedora 6. The algorithms were written in C and compiled with the gcc compiler.

In our experiments, all texts and patterns were

randomly generated. We use the overall execution time to measure the performance of the algorithms. For each combination, a pattern of length m and a text of length n are sampled 100 times with σ symbols. To measure the accuracy of execution time, the times are averaged over 10 runs using the same texts and patterns.

9.1 An Experiment with Random Text Searching for Single Circular Pattern Matching

In our first experiment, we implemented the CBNDM_q algorithm and compared it with the Brute-Force algorithm, the MultiBNDM algorithm [8] and the CBNDM algorithm [7]. We set parameters $n = 10^6$, $m \in \{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\}$ and the alphabet size $\sigma \in \{2, 4, 8, 16\}$.

The set of tested algorithms include several classical algorithms. Besides Brute Force and MultiBNDM [8, 9], we have the source code of CBNDM [7]. We implemented CBNDM_q into two versions: CBNDM_2 for $q = 2$ and $\text{CBNDM}_{2 \log m}$ for $q = 2 \log m$.

The results are shown in Table 1, which lists the execution time of the algorithms. For each alphabet size, CBNDM and its variations, CBNDM_2 and $\text{CBNDM}_{2 \log m}$ are faster than other algorithms. In addition, $\text{CBNDM}_{2 \log m}$ is the fastest for pattern lengths 50-100. Its running times are in the range of 3% – 51.4% of that of CBNDM.

9.2 An Experiment with Random Text Searching for Multiple Circular Patterns Matching

On MCPM, we used tests of size 1 Mb over which we search many circular patterns. We set $r \in \{20, 40, 60, 80, 100\}$ and $m \in \{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\}$. We also performed three kinds of alphabet which is $\sigma \in \{2, 4, 16\}$.

In our test, we have included CMultiBNDM , CMultiBNDM_2 and CMultiBNDM_q where $q = \min \{\lceil \log_\sigma rm \rceil, m\}$. Table 2, table 3 and table 4 show the results for random text with $\sigma = 2, 4, 16$ respectively. In these three tables, CMultiBNDM_q is faster than other algorithms. From these tests, it shows that the filtration algorithms have good behaviors.

10 Conclusion

We propose three algorithms in this paper:

- CBNDM_q for the CPM problem based on bit-parallelism and the q -gram technique;
- CMultiBNDM for the MCPM problem based on bit-parallelism;
- CMultiBNDM_q for the MCPM problem based on bit-parallelism and the q -gram technique.

Theoretically, we show that the optimal value of q should be no more than $\lceil (2 + \epsilon) \log_\sigma m \rceil$ where $0 < \epsilon < 1$ for CBNDM_q . Similarly, it can be shown that $\lceil (2 + \epsilon) \log_\sigma rm \rceil$ is an upper bound of q for CMultiBNDM_q .

In practical consideration, our experiments in Section 9 show that the q -gram technique can boost the basic bit-parallelism algorithms (i.e. CBNDM and CMultiBNDM) dramatically for random texts and patterns, especially when setting $q = \min \{\lceil \log_\sigma rm \rceil, m\}$. However, we do not mean that this is the optimal choice for q . A more complete study on how to choose appropriate values of q for CBNDM_q and CMultiBNDM_q will be invested in the future.

References

- [1] Y. Shiloach. Fast canonization of circular strings. *Journal of Algorithms*, 2(2):107 – 121, 1981.
- [2] D. Gusfield. *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.
- [3] J.-P. Allouche and J. Shallit. *Automatic Sequences – Theory, Applications, Generations*. Cambridge University Press, 2003.
- [4] C. Iliopoulos and M. Rahman. Indexing circular patterns. In Shin-ichi Nakano and Md. Rahman, editors, *WALCOM: Algorithms and Computation*, volume 4921 of *Lecture Notes in Computer Science*, pages 46–57. Springer Berlin / Heidelberg, 2008.
- [5] M. Lothaire. *Applied Combinatorics on Words*. Cambridge University Press, 2005.
- [6] M. Crochemore, C. Hancart, and T. Lecroq. *Algorithms on Strings*. Cambridge University Press, 2007.

Table 1: The execution time (in milliseconds) of the algorithms (Brute Force(A), MultiBNDM (B), CBNDM (C), CBNDM₂ (D), CBNDM_{2 log m} (E))

m	$\sigma = 2$					$\sigma = 4$					$\sigma = 8$					$\sigma = 16$				
	A	B	C	D	E	A	B	C	D	E	A	B	C	D	E	A	B	C	D	E
10	128.0	75.5	32.2	32.0	40.1	84.9	30.5	13.3	14.1	15.7	58.4	22.0	9.1	10.8	12.4	47.4	17.2	7.4	10.0	10.0
20	256.8	101.1	14.0	13.7	14.4	169.6	49.1	7.3	7.4	7.7	116.7	36.4	5.1	5.6	5.7	94.7	29.4	4.2	5.0	5.6
30	384.4	134.8	9.4	9.0	8.8	254.2	69.7	5.2	5.2	5.0	177.5	50.6	3.6	3.9	4.3	141.9	41.5	3.0	3.4	3.7
40	512.6	171.5	11.0	9.6	6.7	338.8	88.3	5.8	5.2	4.0	234.2	64.2	4.1	3.8	3.2	189.2	52.7	3.4	2.8	2.9
50	641.0	214.1	8.9	7.9	5.7	423.7	113.0	4.9	4.3	3.2	291.7	80.4	3.4	3.2	2.6	236.4	66.3	2.8	2.4	2.3
60	769.1	257.1	7.7	6.8	4.7	508.2	135.1	4.2	3.7	2.7	350.0	96.4	3.0	2.7	2.2	283.6	79.3	2.4	2.1	2.0
70	897.3	301.8	8.5	7.8	4.3	592.5	158.7	4.7	4.1	2.6	408.3	113.6	3.4	2.8	2.3	330.8	92.8	2.8	2.0	2.0
80	1026.8	347.6	7.6	7.1	3.8	678.2	181.1	4.2	3.7	2.3	466.7	130.8	3.1	2.5	2.0	377.9	106.5	2.5	1.8	1.8
90	1154.2	390.9	6.8	6.4	3.3	762.9	207.0	3.9	3.4	2.0	525.2	148.6	2.8	2.3	1.8	425.3	120.2	2.2	1.6	1.6
100	1282.7	439.2	7.9	7.3	3.1	847.1	232.5	4.4	3.7	1.8	583.5	167.3	3.2	2.4	1.6	472.5	135.0	2.6	1.6	1.5

 Table 2: The execution time (in milliseconds) of the algorithms (CMultiBNDM (A), CMultiBNDM₂ (B), CMultiBNDM_q (C) where $q = \min \{ \lceil \log_{\sigma} rm \rceil, m \}$) for random text ($\sigma = 2$).

m	$r = 20$			$r = 40$			$r = 60$			$r = 80$			$r = 100$		
	A	B	C	A	B	C	A	B	C	A	B	C	A	B	C
10	1057	787	203	2703	1938	303	4436	3176	157	6160	4379	183	8748	6137	180
20	344	268	49	747	607	94	1274	1027	124	1745	1393	189	2364	1910	293
30	276	217	28	592	475	52	961	775	104	1292	1036	104	1692	1367	158
40	328	261	35	713	575	66	1115	905	83	1544	1262	135	1998	1629	201
50	323	261	39	679	550	72	1056	860	92	1463	1200	151	1894	1555	148
60	308	249	25	652	525	51	1041	855	102	1432	1172	114	1832	1505	167
70	301	246	27	649	525	56	1021	835	78	1402	1148	128	1798	1483	189
80	297	241	29	639	520	62	1005	825	87	1380	1140	142	1768	1482	211
90	296	239	31	636	518	66	998	819	96	1373	1132	157	1753	1459	190
100	292	237	33	630	513	72	995	819	107	1362	1128	174	1745	1448	215

- [7] K. H. Chen, G. S. Huang, and R. C. T. Lee. Exact circular pattern matching using the bndm algorithm. In *Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory*, pages 152–161, 2011.
- [8] G. Navarro and M. Raffinot. Fast and flexible string matching by combining bit-parallelism and suffix automata. *ACM Journal of Experimental Algorithmics*, 5(4), 2000.
- [9] G. Navarro and M. Raffinot. *Flexible pattern matching in strings: Practical on-line search algorithms for texts and biological sequences*. cambridge University Press, 2002.
- [10] A. V. Aho and M. J. Corasick. Efficient string matching: an aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, 1975.
- [11] B. Commentz-Walter. A string matching algorithm fast on the average. In *Proceedings of the 6th Colloquium, on Automata, Languages and Programming*, pages 118–132, London, UK, 1979. Springer-Verlag.
- [12] R. A. Baeza-Yates and G. H. Gonnet. A new approach to text searching. *Communications of the ACM*, 35(10):74–82, 1992.
- [13] S. Wu and U. Manber. A fast algorithm for multi-pattern searching. Technical report, Department of Computer Science, Chung-Cheng University, 1994.
- [14] M. Raffinot. On the multi backward dawg matching algorithm (multibdm). In *R. Baeza-Yates, Editor, Proc. 4th South American Workshop on String Processing*, Carleton University Press, pages 149–165, 1997.
- [15] M. Crochemore and W. Rytter. *Text Algorithms*. Oxford University Press, 1994.
- [16] M. O. Külekci. Tara: An algorithm for fast searching of multiple patterns on text files.

Table 3: The execution time (in milliseconds) of the algorithms (CMultiBNDM (A), CMultiBNDM₂ (B), CMultiBNDM_q (C) where $q = \min \{ \lceil \log_{\sigma} rm \rceil, m \}$ for random text ($\sigma = 4$).

m	$r = 20$			$r = 40$			$r = 60$			$r = 80$			$r = 100$		
	A	B	C	A	B	C	A	B	C	A	B	C	A	B	C
10	160	96	32	340	210	38	529	330	66	734	463	98	963	591	137
20	122	74	13	252	157	33	396	251	25	535	338	37	689	498	57
30	114	71	14	239	150	15	365	235	28	496	325	44	635	417	64
40	139	86	16	293	186	18	457	298	35	630	412	60	813	533	84
50	140	86	17	293	187	21	456	298	41	627	418	67	806	539	49
60	139	87	8	291	189	24	452	299	46	620	414	38	799	536	58
70	136	86	8	292	191	27	455	303	27	623	421	46	791	537	67
80	137	86	9	290	191	29	455	304	32	621	422	54	814	543	79
90	140	88	10	296	193	32	462	306	37	631	425	61	807	550	91
100	140	89	11	295	194	35	459	308	42	625	430	70	798	551	103

 Table 4: The execution time (in milliseconds) of the algorithms (CMultiBNDM (A), CMultiBNDM₂ (B), CMultiBNDM_q (C) where $q = \min \{ \lceil \log_{\sigma} rm \rceil, m \}$ for random text ($\sigma = 16$).

m	$r = 20$			$r = 40$			$r = 60$			$r = 80$			$r = 100$		
	A	B	C	A	B	C	A	B	C	A	B	C	A	B	C
10	61	16	16	126	35	8	191	53	13	257	73	18	324	91	24
20	55	16	4	113	34	8	171	51	14	226	69	21	287	94	30
30	54	16	4	108	33	9	166	54	17	221	75	26	281	100	37
40	66	17	4	134	37	10	207	64	20	283	93	32	365	128	45
50	67	18	4	134	41	13	209	69	23	304	102	37	370	137	17
60	66	18	5	135	42	14	212	73	27	292	107	15	374	144	22
70	65	19	5	137	44	15	215	76	11	294	113	18	378	149	28
80	65	18	6	137	46	17	216	79	13	297	116	23	381	154	34
90	66	20	6	139	48	18	218	83	16	299	120	27	383	173	42
100	66	20	7	140	50	20	219	85	19	302	122	32	386	162	49

In *Computer and information sciences, 2007. iscis 2007. 22nd international symposium on*, pages 1–6, Nov. 2007.

- [17] M. Crochemore, A. Czumaj, L. Gasieniec, T. Lecroq, W. Plandowski, and W. Rytter. Fast practical multi-pattern matching. *Information Processing Letters*, 71(3-4):107 – 113, 1999.
- [18] L. Salmela, J. Tarhio, and J. Kytöjoki. Multi-pattern string matching with q-grams. *Journal of Experimental Algorithmics*, 11:1–19, 2006.
- [19] K. Fredriksson and S. Grabowski. Average-optimal string matching. *Journal of Discrete Algorithms*, 7:579–594, 2009.
- [20] M. Crochemore, A. Czumaj, L. Gasieniec, S. Jarominek, T. Lecroq, W. Plandowski, and W. Rytter. Speeding up two string-matching

algorithms. *Algorithmica*, 12(4-5):247–267, 1994.

- [21] B. Āurian, J. Holub, H. Peltola, and J. Tarhio. Improving practical exact string matching. *Information Processing Letters*, 110(4):148 – 152, 2010.
- [22] C. Miao, G. Chang, and X. Wang. Filtering based multiple string matching algorithm combining q-grams and bndm. In *Proceedings of the 2010 Fourth International Conference on Genetic and Evolutionary Computing*, ICGEC ’10, pages 582–585, Washington, DC, USA, 2010. IEEE Computer Society.
- [23] A. C. Yao. The complexity of pattern matching for a random string. *SIAM Journal on Computing*, 8(3):368–387, 1979.
- [24] D. E. Knuth, J. H. Morris Jr, and V. R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977.